UNIT III

Constructors, Destructors and Operator Overloading

Complete Solutions • Debugging • Structured Pseudocode • Dry Runs

SECTION A — DEBUGGING QUESTIONS

Question 1 — Copy constructor passed by value

1. Compiler error / diagnosis

The declaration DynamicString(DynamicString source) is not a valid copy-constructor declaration. A copy constructor must take its class type by reference (possibly with cv-qualification); a by-value parameter would itself need to be copy-constructed to enter the function, requiring the same constructor recursively. Typical compilers report a diagnostic similar to "invalid constructor; you probably meant DynamicString (const DynamicString&)" or "copy constructor must pass its first argument by reference." Exact wording is compiler-specific.

2. Why the recursion occurs

To initialize the parameter source by value, the language would have to copy the source object. That copy would call DynamicString(DynamicString), whose parameter again requires a copy, and so on. C++ therefore rejects the declaration rather than allowing an infinite chain of copies.

3. Correct signature

```
DynamicString(const DynamicString& source)
```

The reference (&) avoids making a copy just to pass the argument. const allows copying from const objects and temporaries while promising not to modify the source.

4. Complete corrected program

```
#include <iostream>
#include <cstring>

class DynamicString {
private:
    char* buffer;
    int length;

public:
    explicit DynamicString(const char* str) {
        length = static_cast<int>(std::strlen(str));
        buffer = new char[length + 1];
        std::strcpy(buffer, str);
    }

    DynamicString(const DynamicString& source) {
        length = source.length;
        buffer = new char[length + 1];       // fresh allocation
        std::strcpy(buffer, source.buffer); // copy characters
    }

    ~DynamicString() {
        delete[] buffer;
    }

    void print() const {
        std::cout << buffer << '\n';
    }
};

int main() {
    DynamicString s1("Antigravity C++");
    DynamicString s2 = s1;
    s2.print();
}
```

Question 2 — Shallow copy and double free

1. What the synthesized copy constructor does

The compiler-generated copy constructor performs memberwise copying. It copies capacity as an integer and samples as a pointer value. Thus track.samples and buf.samples contain the same address; no second array is allocated.

Stage	track.samples	buf.samples	Ownership state
After AudioBuffer track(5)	Address A	—	track owns array A
During processAudio(track)	Address A	Address A	Both objects point to A
When buf is destroyed	Address A (now dangling)	Address A	buf deletes A
When track is destroyed	Address A (dangling)	—	Second delete of A: undefined behavior

2. Cause of the crash

The local parameter buf is destroyed when processAudio returns and deletes the shared array. track still stores the same now-invalid pointer. At the end of main, track's destructor attempts to delete that allocation again. This is a double deletion and undefined behavior; a runtime allocator may report “double free detected,” but the exact message is not guaranteed.

3. Deep-copy constructor

```
AudioBuffer(const AudioBuffer& other) : capacity(other.capacity) {
    samples = new int[capacity];
    for (int i = 0; i < capacity; ++i) {
        samples[i] = other.samples[i];
    }
}
```

A complete corrected class and driver

```
#include <iostream>

class AudioBuffer {
public:
    int* samples;
    int capacity;

    explicit AudioBuffer(int cap) : capacity(cap) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i) samples[i] = i * 10;
        std::cout << "Allocated " << samples << '\n';
    }

    AudioBuffer(const AudioBuffer& other) : capacity(other.capacity) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i) samples[i] = other.samples[i];
        std::cout << "Deep copy allocated " << samples << '\n';
    }

    ~AudioBuffer() {
        std::cout << "Deleting " << samples << '\n';
        delete[] samples;
    }
};

void processAudio(AudioBuffer buf) {
    std::cout << "Processing sample 0: " << buf.samples[0] << '\n';
}

int main() {
    AudioBuffer track(5);
    processAudio(track);
    std::cout << "Back in main, sample 0: " << track.samples[0] << '\n';
}
```

4. Output interpretation

The following is an illustrative run. Heap addresses vary each time and between systems; the important fact is that the two

allocation addresses are distinct.

```
Allocated 0x...A
Deep copy allocated 0x...B
Processing sample 0: 0
Deleting 0x...B
Back in main, sample 0: 0
Deleting 0x...A
```

Question 3 — Stream insertion operator must be non-member

1. Member-operator translation

For a binary member operator, left @ right is interpreted as left.operator@(right). Therefore std::cout << f1 attempts std::cout.operator<<(f1). Fraction's member operator is not considered as the left operand is an ostream, not a Fraction.

2. Why the reversed form compiles

f1 << std::cout becomes f1.operator<<(std::cout), which matches the student's member signature. It reverses the conventional stream syntax and is not suitable for normal output expressions or chaining.

3–4. Correct friend operator and program

```
#include <iostream>

class Fraction {
private:
    int numerator;
    int denominator;

public:
    Fraction(int n, int d) : numerator(n), denominator(d) {}

    friend std::ostream& operator<<(std::ostream& os,
                                    const Fraction& value) {
        os << value.numerator << "/" << value.denominator;
        return os;
    }
};

int main() {
    Fraction f1(3, 4);
    std::cout << f1 << '\n';
}
```

Expected output

```
3/4
```

Question 4 — Prefix and postfix increment

1. Canonical signatures

Form	Signature	Return	Meaning
Prefix ++obj	StepCounter& operator++()	Reference	Increment first; return modified object
Postfix obj++	StepCounter operator++(int)	Value	Save old value, increment object, return old snapshot

2. Dummy int parameter

The unused int parameter distinguishes postfix from prefix because both use the ++ token. The compiler supplies this dummy argument for the postfix form; the caller does not write it.

3. Return-type rationale

Prefix returns a reference to the updated object, avoiding an unnecessary copy and allowing expressions such as ++(++c). Postfix must preserve the old state, so it returns a copy by value.

4. Corrected code

```
#include <iostream>

class StepCounter {
private:
    int steps;

public:
    explicit StepCounter(int s = 0) : steps(s) {}

    StepCounter& operator++() {       // prefix
        ++steps;
        return *this;
    }

    StepCounter operator++(int) {     // postfix
        StepCounter old(*this);
        ++steps;
        return old;
    }

    void display() const {
        std::cout << "Steps: " << steps << '\n';
    }
};

int main() {
    StepCounter c1(10);
    StepCounter c2 = ++c1; // c1 becomes 11; c2 is 11
    c2.display();
    c1.display();

    StepCounter c3 = c1++; // c3 is 11; c1 becomes 12
    c3.display();
    c1.display();
}
```

Expected output

```
Steps: 11
Steps: 11
Steps: 11
Steps: 12
```

Question 5 — Fundamental operator-overloading rules

1. Operators that cannot be overloaded

The operators listed in the question bank are: . (member access), .* (pointer-to-member access), :: (scope resolution), ?: (conditional), and sizeof. The dot operator is part of the language's member-access grammar and cannot be given user-defined behavior.

2. Why int + int cannot be overloaded

An overloaded operator must have at least one operand of a user-defined class or enumeration type. Both operands in int operator+(int, int) are built-in types, so the declaration is prohibited.

3. Why ** cannot be invented

C++ permits overloading only existing operator tokens. A programmer cannot create a new token such as **. Overloading also cannot change an operator's precedence, associativity, or number of operands (arity).

4. Legal replacement using ==, +, and a power method

```
#include <iostream>

class MathVector {
public:
    double x, y;

    MathVector(double a = 0.0, double b = 0.0) : x(a), y(b) {}

    bool operator==(const MathVector& other) const {
        return x == other.x && y == other.y;
    }

    MathVector operator+(const MathVector& other) const {
        return MathVector(x + other.x, y + other.y);
    }

    MathVector power(int exponent) const {
        // Component-wise integer power; exponent must be non-negative.
        double px = 1.0, py = 1.0;
        for (int i = 0; i < exponent; ++i) {
            px *= x;
            py *= y;
        }
        return MathVector(px, py);
    }
};

int main() {
    MathVector a(2.0, 3.0), b(2.0, 3.0);
    MathVector sum = a + b;
    MathVector squared = a.power(2);

    std::cout << (a == b) << '\n';
    std::cout << sum.x << ", " << sum.y << '\n';
    std::cout << squared.x << ", " << squared.y << '\n';
}
```

Expected output

```
1
4, 6
4, 9
```

SECTION B — PSEUDOCODE WRITING QUESTIONS

Question 1 — Dynamic string engine with deep copy and overloaded +

Structured pseudocode

```
CLASS DynStr
    PRIVATE:
        dataPtr : POINTER TO CHARACTER
        charCount : INTEGER

    PUBLIC:
        CONSTRUCTOR(sourceString AS STRING)
            charCount <- LENGTH(sourceString)
            dataPtr <- ALLOCATE ARRAY[charCount + 1] OF CHARACTER
            COPY sourceString INTO dataPtr
            dataPtr[charCount] <- NULL
        END CONSTRUCTOR

        COPY CONSTRUCTOR(REF other AS DynStr)
            charCount <- other.charCount
            dataPtr <- ALLOCATE ARRAY[charCount + 1] OF CHARACTER
            FOR i <- 0 TO charCount - 1
                dataPtr[i] <- other.dataPtr[i]
            END FOR
            dataPtr[charCount] <- NULL
        END COPY CONSTRUCTOR

        DESTRUCTOR()
            DEALLOCATE ARRAY dataPtr
            dataPtr <- NULL
        END DESTRUCTOR

        OPERATOR +(REF rhs AS DynStr) RETURNS DynStr
            total <- charCount + rhs.charCount
            temp <- ALLOCATE ARRAY[total + 1] OF CHARACTER
            COPY dataPtr INTO temp
            APPEND rhs.dataPtr TO temp
            result <- DynStr(temp)
            DEALLOCATE ARRAY temp
            RETURN result
        END OPERATOR

        METHOD DISPLAY()
            PRINT dataPtr
        END METHOD
END CLASS

MAIN
    strA <- DynStr("Hello ")
    strB <- DynStr("World!")
    strC <- strA + strB
    strC.DISPLAY()
END MAIN
```

Dry run: strA = “Hello ” and strB = “World!”

Object / operation	Character count	Allocated buffer	Work
strA	6	7 characters	Copy H e l l o + terminator
strB	6	7 characters	Copy W o r l d ! + terminator
strC = strA + strB	12	13 characters	Copy 6 chars from A, append 6 from B, add terminator

Result: Hello World!. The two source objects each own independent buffers; the result owns a third buffer. The temporary concatenation buffer is released after the result is constructed. A production implementation should also define copy assignment or use a standard string type.

Question 2 — Complex number system

Structured pseudocode

```
CLASS Complex
    PRIVATE:
        real : REAL
        imag : REAL

    PUBLIC:
        CONSTRUCTOR()
            real <- 0.0
            imag <- 0.0
        END CONSTRUCTOR

        CONSTRUCTOR(r AS REAL)
            real <- r
            imag <- 0.0
        END CONSTRUCTOR

        CONSTRUCTOR(r AS REAL, i AS REAL)
            real <- r
            imag <- i
        END CONSTRUCTOR

        OPERATOR +(REF other AS Complex) RETURNS Complex
            RETURN Complex(real + other.real, imag + other.imag)
        END OPERATOR

        OPERATOR *(REF other AS Complex) RETURNS Complex
            newReal <- real * other.real - imag * other.imag
            newImag <- real * other.imag + imag * other.real
            RETURN Complex(newReal, newImag)
        END OPERATOR

        OPERATOR ==(REF other AS Complex) RETURNS BOOLEAN
            RETURN (real = other.real) AND (imag = other.imag)
        END OPERATOR

        METHOD DISPLAY()
            PRINT real, " + ", imag, "i"
        END METHOD
END CLASS

MAIN
    C1 <- Complex(3, 2)
    C2 <- Complex(1, 4)
    product <- C1 * C2
    product.DISPLAY()
END MAIN
```

Dry run: (3 + 2i)(1 + 4i)

Calculation	Value
Real part = (3 × 1) − (2 × 4)	3 − 8 = −5
Imaginary part = (3 × 4) + (2 × 1)	12 + 2 = 14
Product	−5 + 14i

The result is −5 + 14i.

Question 3 — Financial currency class

Structured pseudocode

```
CLASS Currency
    PRIVATE:
        dollars : INTEGER
        cents : INTEGER

    PUBLIC:
        CONSTRUCTOR(d = 0, c = 0)
            totalCents <- d * 100 + c
            dollars <- totalCents DIV 100
            cents <- totalCents MOD 100
        END CONSTRUCTOR

        OPERATOR -() RETURNS Currency
            totalCents <- dollars * 100 + cents
            RETURN Currency(0, -totalCents)
        END OPERATOR

        FRIEND OPERATOR <<(REF outStream, REF cur AS Currency)
            outStream PRINT "$", cur.dollars, ".",
                TWO_DIGIT_FORMAT(cur.cents)
            RETURN outStream
        END OPERATOR

        FRIEND OPERATOR >>(REF inStream, REF cur AS Currency)
            READ d, c FROM inStream
            cur <- Currency(d, c)
            RETURN inStream
        END OPERATOR
END CLASS

MAIN
    cur <- Currency(5, 275)
    PRINT cur
    debt <- -cur
    PRINT debt
END MAIN
```

Normalization trace

Step	Calculation	Normalized result
Input	d = 5, c = 275	Total = 5×100 + 275 = 775 cents
Normalize	775 DIV 100 = 7; 775 MOD 100 = 75	$7.75
Unary negation	−775 cents	−$7.75

The total-cents approach handles cents values above 99 and negative values consistently. In an implementation, formatted negative currency should be printed with a single leading minus sign (for example, -$7.75); the pseudocode's stream formatter should apply that sign convention.

Question 4 — Polynomial with [] and +

Structured pseudocode

```
CLASS Polynomial
    PRIVATE:
        degree : INTEGER
        coeffArray : POINTER TO REAL

    PUBLIC:
        CONSTRUCTOR(deg AS INTEGER)
            degree <- deg
            coeffArray <- ALLOCATE ARRAY[degree + 1] OF REAL
            FOR i <- 0 TO degree
                coeffArray[i] <- 0.0
            END FOR
        END CONSTRUCTOR

        COPY CONSTRUCTOR(REF other AS Polynomial)
            degree <- other.degree
            coeffArray <- ALLOCATE ARRAY[degree + 1] OF REAL
            FOR i <- 0 TO degree
                coeffArray[i] <- other.coeffArray[i]
            END FOR
        END COPY CONSTRUCTOR

        DESTRUCTOR()
            DEALLOCATE ARRAY coeffArray
            coeffArray <- NULL
        END DESTRUCTOR

        OPERATOR [](index AS INTEGER) RETURNS REF REAL
            IF index < 0 OR index > degree
                RAISE INDEX_ERROR
            END IF
            RETURN REFERENCE TO coeffArray[index]
        END OPERATOR

        OPERATOR +(REF other AS Polynomial) RETURNS Polynomial
            maxDegree <- MAX(degree, other.degree)
            result <- Polynomial(maxDegree)
            FOR i <- 0 TO maxDegree
                leftValue <- 0.0
                rightValue <- 0.0
                IF i <= degree
                    leftValue <- coeffArray[i]
                END IF
                IF i <= other.degree
                    rightValue <- other.coeffArray[i]
                END IF
                result[i] <- leftValue + rightValue
            END FOR
            RETURN result
        END OPERATOR
END CLASS

MAIN
    P1 <- Polynomial(2)
    P1[0] <- 5
    P1[1] <- 2
    P1[2] <- 3

    P2 <- Polynomial(3)
    P2[0] <- 7
    P2[1] <- 0
    P2[2] <- 1
    P2[3] <- 4

    P3 <- P1 + P2
    DISPLAY P3
END MAIN
```

Dry run: P1(x) = $3x^2 + 2x + 5$ and P2(x) = $4x^3 + x^2 + 7$

Power	P1 coefficient	P2 coefficient	Sum
x^3	0	4	4
x^2	3	1	4
x^1	2	0	2
x^0	5	7	12

Therefore P3(x) = $4x^3 + 4x^2 + 2x + 12$. Coefficients are stored from index 0 (constant term) through index degree.

Question 5 — Vector2D prefix/postfix ++ and +=

Structured pseudocode

```
CLASS Vector2D
    PRIVATE:
        x : REAL
        y : REAL

    PUBLIC:
        CONSTRUCTOR(a = 0.0, b = 0.0)
            x <- a
            y <- b
        END CONSTRUCTOR

        OPERATOR ++() RETURNS REF Vector2D
            x <- x + 1.0
            y <- y + 1.0
            RETURN REFERENCE TO THIS
        END OPERATOR

        OPERATOR ++(dummy INTEGER) RETURNS Vector2D
            oldValue <- COPY OF THIS
            x <- x + 1.0
            y <- y + 1.0
            RETURN oldValue
        END OPERATOR

        OPERATOR +=(REF rhs AS Vector2D) RETURNS REF Vector2D
            x <- x + rhs.x
            y <- y + rhs.y
            RETURN REFERENCE TO THIS
        END OPERATOR

        METHOD DISPLAY()
            PRINT "(", x, ", ", y, ")"
        END METHOD
END CLASS

MAIN
    v1 <- Vector2D(5.0, 8.0)
    v2 <- ++v1
    v3 <- v1++
    DISPLAY v1, v2, v3
END MAIN
```

Dry run

Statement	v1 after statement	Assigned value
Initial state	(5.0, 8.0)	—
v2 <- ++v1	(6.0, 9.0)	v2 = (6.0, 9.0)
v3 <- v1++	(7.0, 10.0)	v3 = old snapshot (6.0, 9.0)

Final values: v1 = (7.0, 10.0), v2 = (6.0, 9.0), and v3 = (6.0, 9.0). Prefix returns the updated object by reference; postfix returns the saved old value by copy.